

Random Oracles in Constantinople: Practical Asynchronous Byzantine Agreement Using Cryptography (Extended Abstract)

Christian Cachin

Klaus Kursawe

Victor Shoup

IBM Research
Zurich Research Laboratory
CH-8803 Rüschlikon, Switzerland
{cca,kku,sho}@zurich.ibm.com

Abstract

Byzantine agreement requires a set of parties in a distributed system to agree on a value even if some parties are corrupted. A new protocol for Byzantine agreement in a completely asynchronous network is presented that makes use of cryptography, specifically of threshold signatures and coin-tossing protocols. These cryptographic protocols have practical and provably secure implementations in the “random oracle” model. In particular, a coin-tossing protocol based on the Diffie-Hellman problem is presented and analyzed.

The resulting asynchronous Byzantine agreement protocol is both practical and nearly matches the known theoretical lower bounds. More precisely, it tolerates the maximum number of corrupted parties, runs in constant expected time, has message and communication complexity close to the maximum, and uses a trusted dealer only in a setup phase, after which it can process a virtually unlimited number of transactions. Novel dual-threshold variants of both cryptographic protocols are used.

The protocol is formulated as a transaction processing service in a cryptographic security model, which differs from the standard information-theoretic formalization and may be of independent interest.

Keywords: Asynchronous Consensus, Byzantine Faults, Threshold Signatures, Cryptographic Common Coin, Dual-Threshold Schemes.

1 Introduction

The (binary) Byzantine agreement problem is one of the fundamental problems in distributed fault-tolerant computing. In this problem, there are n communicating parties, at most t of which are corrupted. The goal is that all honest (i.e., uncorrupted) parties agree on one of two values that was proposed by an honest party, despite the malicious behavior of the corrupted parties. This problem has been studied under various assumptions regarding the synchrony of the network, the privacy of the communication channels, and the computational power of the corrupted parties.

In this paper, we work exclusively in an *asynchronous* environment with computationally bounded parties; our motivation for this is a secure distributed system connected by the Internet.

Fischer, Lynch, and Paterson [FLP85] have shown that no deterministic protocol can guarantee agreement even against benign failures in the asynchronous setting. Rabin [Rab83] and Ben-Or [Ben83] were the first to present protocols that overcome this limitation by using randomization. They assume a *common coin*, a random source observable by all participants but unpredictable for an adversary; this abstraction is used in most subsequent protocols for the asynchronous model.

Our main contributions are an agreement protocol and a common coin protocol that employ modern cryptographic techniques to a far greater extent than has been done previously in the literature. The basic cryptographic primitives used are a non-interactive threshold signature scheme and a novel threshold, random-access coin-tossing scheme. We use dual-threshold variants of both primitives. They can be efficiently implemented and proved secure under standard intractability assumptions in the *random oracle* model; in this model, one treats a cryptographic hash function as if it were a black box containing a random function.

Taken together, we obtain a new protocol for Byzantine agreement that is both practical and theoretically nearly optimal with respect to the known lower bounds because

- it withstands the maximum number of corrupted parties: $t < n/3$;
- it runs in constant expected time;
- the expected number of messages is $O(n^2)$;
- each message is roughly the size of one or two RSA signatures (with the RSA threshold signature scheme of Shoup [Sho00]);
- it uses a trusted dealer only in a setup phase, after which it can process a virtually unlimited number of transactions.

This last point deserves further elaboration. The initial setup phase of our scheme requires a trusted dealer to distribute certain cryptographic keys. Once in place, however, our scheme provides a *transaction processing service* that can handle a virtually *unlimited* number of requests as generated by clients. Moreover, transactions can be processed

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

PODC 2000 Portland Oregon

Copyright ACM 2000 1-58113-183-6/00/07...\$5.00

concurrently, i.e., a new instance of the agreement protocol can start as soon as a new transaction request is generated by a client, even if there are extant instances of the protocol for other transactions. This is a non-trivial but important feature for any cryptographic protocol because it rules out so-called interleaving attacks.

1.1 Techniques

Our protocol uses non-interactive threshold signatures and a random-access coin-tossing scheme from cryptography; these have efficient implementations in the random oracle model.

The *random oracle model* was first used in a rather informal way by Fiat and Shamir [FS87]; it was first formalized and used in other contexts by Bellare and Rogaway [BR93] and has since been used to analyze a number of practical cryptographic protocols. Of course, it would be better not to rely on random oracles, as they are essentially a heuristic device; nevertheless, random oracles are a useful tool—they allow us to design truly practical protocols that admit a security analysis, which yields very strong evidence for their security. As far as we know, our work is the first of its kind to apply the random oracle model to the Byzantine agreement problem.

The notion of a *threshold signature scheme* was introduced by Desmedt, Frankel and others [Des88, DF90, Boy89, CH89] and has been widely studied since then (T. Rabin [Rab98] provides new results and a survey of recent literature). It is a protocol for n parties tolerating up to t corruptions, where each party holds a share of the signing key and k cooperating parties together can generate a signature. In a *non-interactive* threshold signature scheme, each party outputs a signature share upon request and there is an algorithm to combine k valid signature shares to constitute a valid signature. Such non-interactive combination is used in our agreement protocol: a party can justify its vote for a particular value by a single threshold signature generated from k signature shares. This saves a factor n in terms of bit complexity.

One of the technical contributions of this paper is the notion of a *dual-threshold signature scheme*, meaning that k is allowed to be higher than $t + 1$. This is in contrast to all previous work on threshold signatures in the literature where $k = t + 1$. A companion paper [Sho00] presents a practical dual-threshold signature scheme that is secure in the random oracle model under standard intractability assumptions. The signatures created by this scheme are ordinary RSA signatures. Moreover, the scheme is completely non-interactive, an individual share of a signature is not much greater than an ordinary RSA signature, and even for $k = t + 1$, it is the first rigorously analyzed non-interactive threshold signature scheme with small shares.

Coin-tossing schemes are used in one form or another in essentially all solutions to the asynchronous Byzantine agreement problem. Many schemes, following Rabin's pioneering work [Rab83], assume that coins are pre-distributed (and possibly signed) by a dealer using secret-sharing [Sha79]. This approach has two problems: first, the coins will eventually be exhausted; second, parties must somehow associate coins with transactions, which itself represents an agreement problem. Because of these problems, protocols that rely on a "Rabin dealer" are not really suitable for use as a transaction processing service as described here. The same applies to the coin-tossing scheme of Beaver

and So [BS94], which essentially gives parties sequential access to a bounded number of coins. A "Rabin dealer" has been used in other contexts as well, e.g., for threshold decryption [CG99]. Our protocol also requires a dealer for the initial setup, but yields an arbitrary polynomial number of coins afterwards.

The beautiful work of Canetti and T. Rabin [CR93] presents a coin-tossing scheme that allows common coins to be generated entirely "from scratch," building on the work of Feldman and Micali for the synchronous model [FM97]. Unfortunately, this scheme, while polynomial time, is completely impractical.

Our approach to coin-tossing is to use a random-access coin-tossing scheme—essentially a distributed function mapping the "name" of a coin to its value. Such coin-tossing schemes have been studied before [MS95, NPR99]. We also define the notion of a *dual-threshold coin-tossing scheme*, which is convenient and does lead to lower communication complexity, but is not absolutely necessarily. One could easily implement such a coin from the non-interactive threshold signature scheme of Shoup [Sho00]; however, we present a dual-threshold coin-tossing scheme that is based on the Diffie-Hellman problem, the analysis of which may be interesting in its own right. This scheme is essentially the same as the one of Naor et al. [NPR99], but our analysis is more refined: first, for the single-parameter setting, we need a weaker intractability assumption, and second, we provide an analysis of the scheme in the dual-threshold setting, which is not considered by Naor et al.

We stress that such dual-parameter threshold schemes provide stronger security guarantees than single-parameter threshold schemes, and they are in fact more challenging to construct and to analyze. Our notion of a dual-threshold scheme should not be confused with a weaker notion that sometimes appears in the literature (e.g., [MS95]). For this weaker notion, there is a parameter $l > t$ such that the reconstruction algorithm requires l shares, but the security guarantee for a given signature/coin is lost if just a *single* honest party reveals a share. In our notion, no security is lost unless $k - t$ honest parties reveal their shares.

1.2 Related Work

The problem of asynchronous Byzantine agreement has a long history—see the survey of the early Byzantine era by Chor and Dwork [CD89] and the more recent account by Berman and Garay [BG93]. A fundamental result in this area is the impossibility result of Fischer, Lynch, and Paterson [FLP85] that rules out the existence of a deterministic protocol. The protocols of Rabin [Rab83] and Ben-Or [Ben83] are the first probabilistic protocols to overcome this limitation. Bracha's protocol improves the resilience to the maximum $t < n/3$ [Bra84]. Numerous protocols (including ours) build on these [Tou84, FM97, BG93, CR93], but a detailed comparison of our approach with them is beyond the scope of this extended abstract. Below we briefly discuss the most important parameters.

Our protocol achieves the maximum possible *resilience* $t < n/3$ like those of Toueg [Tou84], Bracha [Bra84], and Canetti and Rabin [CR93]. It runs in *constant expected time* like the protocols by Toueg [Tou84], Canetti and Rabin [CR93], and Berman and Garay [BG93].

The expected *message complexity* is $O(n^2)$; all other protocols in the literature require either more messages [BG93, Tou84, CR93] or do not achieve optimal resilience [BG93].

Dolev and Strong establish a lower bound of $\Omega(nt)$ messages for (deterministic) authenticated protocols [DS83]. With RSA-based threshold signatures, our protocol has *bit complexity* $O(n^2l)$, where l is the length of an RSA signature. This is an improvement by a factor of n compared to the most efficient asynchronous protocol so far, Bracha's protocol [Bra84] combined with signatures. Because of the public-key operations, the *computational complexity* of our protocol is typically higher than those using authentication codes [BG93].

We note that our protocol requires a trusted *dealer* only for initial setup; other, less efficient protocols [Bra84, CR93] require no dealer at all. Protocols based on a "Rabin dealer" predistribute data for each transaction [Tou84, BG93].

Our adversary is *computationally bounded* (in order for digital signatures to be meaningful) and chooses who to corrupt independent of the network traffic (i.e., statically).

The only other work in the asynchronous Byzantine fault model with a practical focus is by Reiter [Rei96], who adapts failure detectors [CT96] from the asynchronous crash-failure model and obtains an efficient solution for broadcast and agreement. Most implementations of failure detectors seem to require some timing assumption, however, and as Reiter's protocol is deterministic, the impossibility result [FLP85] implies that it can not solve asynchronous Byzantine agreement.

1.3 Organization

In §2 we introduce our asynchronous system model using cryptography. §3 contains the definition of Byzantine agreement and §4 introduces the cryptographic primitives of threshold signatures and coin-tossing protocols. The agreement protocol based on these primitives is presented in §5 and our coin-tossing protocol is given in §6. Some proofs have been omitted for lack of space and can be found in the full version [CKS00].

2 Basic System Model

In this section, we describe our basic system model for an arbitrary multi-party protocol where a number of parties communicate over an insecure, asynchronous network, and where an adversary may corrupt some of the parties. Our point of view is *computational*: all parties and the adversary are constrained to perform only feasible computations. This differs substantially from the traditional secure channels model in distributed computing, but is necessary and also appropriate for the cryptographic setting (cf. [BR95, BCK98, Sho99]). Although authentication and digital signatures have been used before in agreement protocols, there seem to be no adequate cryptographic formal models [Lyn96, p. 115].

There are n parties, $P_1, \dots, P_n$, an adversary that is allowed to corrupt up to t of them, and a trusted "dealer."

We adopt the *static* corruption model, wherein the adversary must decide whom to corrupt at the very outset of the execution of the system. Let f , with $0 \leq f \leq t$, denote the number of parties the adversary actually corrupts. These corrupted parties are simply absorbed into the adversary: we do not regard them as system components.

Alternatively, one could adopt the *adaptive* corruption model, wherein the adversary can adaptively choose whom to corrupt as the attack is ongoing, based on information

it has accumulated so far. We do not adopt this model, mainly because we would no longer know how to obtain the practical, provably secure implementations of the necessary cryptographic primitives. Moreover, the static corruption model is not too unrealistic; in practice, the choice of whom to corrupt is usually based on factors totally independent of the network traffic (e.g., which system administrator is not careful, or can perhaps be bribed or blackmailed).

There is an initial setup phase, in which the trusted dealer generates the initial state for all n parties. The adversary obtains the initial state of the corrupted parties, but obtains no information about the initial state given to the honest parties.

Our network is insecure and asynchronous, i.e., the adversary has complete control of the network: he may schedule the delivery of messages as he wishes, and may modify or insert messages as he wishes. As such, the network is merely absorbed into the adversary in our formal model. The honest parties are completely passive: they simply *react* to requests made by the adversary and maintain their internal state between requests. More precisely, after the initial setup phase, the adversary performs a number of *basic steps*. One basic step works as follows: the adversary delivers a message to an honest party P_i ; then P_i updates its internal state, and computes a set of response messages; these messages are then given to the adversary. These response messages perhaps indicate to whom these messages should be sent, and the adversary may choose to deliver these messages faithfully at some time. In general, the adversary chooses to deliver any messages it wants, or no messages at all; we may sometimes impose additional restrictions on the adversary's behavior, however.

Of course, the computations made by the honest parties, the adversary, and the dealer should all be representable as probabilistic, polynomial-time computations. To be completely formal, we would have to introduce a security parameter, and all the computations would be bounded by a polynomial in this security parameter. In particular, the parameter n and the number of basic steps performed by the adversary are polynomially bounded in the security parameter.

The dealing algorithm and the algorithm executed locally by each P_i to compute its new state and response messages are specific to the particular protocol. The dealing algorithm is given the security parameter, as well as n and t as input. Note that the adversary chooses n and t , but a specific protocol might impose its own restrictions (e.g., $t < n/3$). We can assume that the dealer includes these values, as well as the index i , in the initial state of P_i .

3 Definition of Byzantine Agreement

We now define the operation and requirements of a Byzantine agreement protocol, in the context of our basic system model described in the previous section. There are n parties, $P_1, \dots, P_n$, and the adversary may corrupt some number f of them, where $f \leq t$.

As mentioned in the introduction, we want an agreement protocol that can be used to implement a *transaction processing service*. To this end, we assume that each decision to be made is associated with a unique transaction identifier *TID*. The value *TID* is an arbitrary bit string whose structure and meaning are determined by a particular application. In our formal model, it is simply chosen by the

adversary.

The adversary may deliver a message to P_i of the form

$$(TID, \text{activate}, \text{initial value}),$$

where *initial value* is in $\{0, 1\}$. When the adversary has delivered such a message, we say that P_i is *activated* on TID with the given *initial value*. After activating P_i on TID , the adversary may then deliver messages to P_i of the form

$$(TID, j, i, \dots),$$

where $1 \leq j \leq n$ denotes the index of the sender.

Upon receiving a message involving TID , P_i updates its internal state, and generates a set (possibly empty) of response messages. Each of these messages is either of the form

$$(TID, i, j, \dots),$$

where $1 \leq j \leq n$ denotes the index of the recipient, or

$$(TID, \text{decide}, \text{final value}),$$

where *final value* $\in \{0, 1\}$. In the latter case, we say that P_i *decides final value* for TID . We require that P_i makes a decision for a given TID at most once. However, the adversary may continue to deliver messages involving TID after P_i has made a decision for TID .

For simplicity, we shall assume that messages are *authenticated*, which means that we restrict the adversary's behavior as follows: if P_i and P_j are honest, and the adversary delivers a message M of the form $(TID, i, j, \dots)$ to P_j , then the message M must have been generated by P_i at some prior point in time. It is reasonable to build authentication into our model because it can be implemented very cheaply using standard symmetric-key cryptographic techniques.

The three basic properties that an agreement protocol must satisfy are *agreement*, *termination*, and *validity*.

Agreement. Any two honest parties that decide a value for a particular TID must decide the same value. More precisely, it is computationally *infeasible* for an adversary to make two honest parties decide on different values.

Termination. The traditional approach in the distributed computing literature is to assume that all messages between honest parties are “eventually” delivered, and then to define the termination condition to be that all honest parties “eventually” decide (with probability 1). In formalizing these definitions, one considers infinite runs of a protocol; however, in the computationally bounded setting, this simply does not work.

We present here a workable definition in our setting that captures the intuition that *to the extent* the adversary delivers messages among honest parties, the honest parties *quickly* decide. Although the intuition is fairly clear, one has to be careful with the details. For us, *termination* consists of two conditions: *deadlock freeness* and *fast convergence*.

Deadlock freeness. It is infeasible for the adversary to create a situation where for some TID there are some honest parties who are not decided, yet all honest parties have been activated on TID , and all messages relating to this TID generated by honest parties have been delivered.

Fast Convergence. For $s = 1, 2, \dots$, let TID_s denote the s th transaction identifier introduced by the adversary, and

define X_s to be the total number of messages *generated* by all honest parties that relate to TID_s . Then there exist fixed polynomials B and C in n and in the security parameter such that for all $s \geq 1$ and $m \geq 1$,

$$\Pr[X_s \geq mB + C] \leq 2^{-m} + \epsilon,$$

where ϵ is a function that is negligible in the security parameter (i.e., it vanishes faster than any polynomial in the security parameter). Note that while ϵ may depend on the adversary, the polynomials B and C depend only on the agreement protocol, and are independent of the adversary.

The *deadlock freeness* property rules out trivial protocols that never decide and never generate any messages to be delivered. The *fast convergence* property ensures timely convergence, provided the adversary delivers messages; also, the fact that B and C are independent of the adversary rules out trivial protocols that never decide but always generate “make work” messages to be delivered.

Our definition of *termination* implies that an adversary *could* quickly make all honest parties make a decision on a given TID (with probability exponentially close to 1) by delivering a (fixed) polynomially bounded number of messages; however, we do not *force* the adversary to do so—see [Can96] for a definition more along these lines.

Validity. If all honest parties are activated on a given TID with the same initial value, then any honest party that decides must decide this value.

This is the usual definition of validity in the literature. A weaker notion of validity may sometimes be more appropriate for particular applications. For instance, initial values may come with validating data (e.g., a digital signature) that establishes the “validity” of a value in a particular context. One could then simply require that an honest party may only decide on a value for which it has the accompanying validating data—even if all honest parties start with 0, they may still decide on 1 if they obtain the corresponding validating data for 1 during the agreement protocol.

4 Cryptographic Primitives

4.1 Digital Signatures

A digital signature scheme [GMR88] consists of a *key generation* algorithm, a *signing* algorithm, and a *verification* algorithm. The key generation algorithm takes as input a security parameter, and outputs a public key/private key pair (PK, SK) . The signing algorithm takes as input SK and a message M , and produces a signature σ . The verification algorithm takes PK , a message M , and a putative signature σ , and outputs either *accept* or *reject*. A signature is considered *valid* if and only if the verification algorithm accepts. All signatures produced by the signing algorithm must be valid.

The basic security property is *unforgeability*. The attack scenario is as follows. An adversary is given the public key, and then requests the signatures on a number of messages, where the messages themselves may depend on previously obtained signatures. If at the end of the attack, the adversary can output a message M and a valid signature σ on M , such that M was not one of the messages whose signature he requested, then the adversary has successfully *forged* a signature. Security means that it is computationally infeasible for an adversary to forge a signature.

4.2 Threshold Signatures

In this section, we define the notion of an (n, k, t) dual-threshold signature scheme. The basic idea is that there are n parties, up to t of which may be corrupted. The parties hold *shares* of the secret key of a signature scheme, and may generate *shares of signatures* on individual messages— k signature shares are both necessary and sufficient to construct a signature. The only requirement on k is that $t < k \leq n - t$. As mentioned in the introduction, previous investigations into threshold signatures have only considered the case $k = t + 1$. Also, we shall require that the generation and verification of signature shares is completely non-interactive—this is essential in the application of asynchronous Byzantine agreement.

A threshold signature scheme is a multi-party protocol, and we shall work in our basic system model for such protocols (see §2).

The Action. The dealer generates a public key PK along with secret key shares $SK_1, \dots, SK_n$, a global verification key VK , and local verification keys $VK_1, \dots, VK_n$. The initial state information for party P_i consists of the secret key SK_i along with the public key and *all* the verification keys.

After the dealing phase, the adversary submits signing requests to the honest parties for messages of his choice. Upon such a request, party P_i computes a *signature share* for the given message using SK_i .

Combining Signature Shares. The threshold signature scheme also specifies three algorithms: a *signature verification* algorithm, a *share verification* algorithm, and a *share combining* algorithm.

- The *signature verification* algorithm takes as input a message and a signature (generated by the share-combining algorithm), along with the public key, and determines if the signature is valid.
- The *share verification* algorithm takes as input a message, a signature share on that message from a party P_i , along with PK , VK , and VK_i , and determines if the signature share is valid.
- The *share combining* algorithm takes as input a message and k valid signature shares on the message, along with the public key and (perhaps) the verification keys, and (hopefully) outputs a valid signature on the message.

Security Requirements. The two basic security requirements are *robustness* and *non-forgeability*.

Robustness. If it is computationally infeasible for an adversary to produce k valid signature shares such that the output of the share combining algorithm is not a valid signature.

Non-forgeability. It is computationally infeasible for the adversary to output a valid signature on a message that was submitted as a signing request to less than $k - t$ honest parties. Note that if the adversary actually corrupts $f < t$ parties, the relevant threshold is still $k - t$ and not $k - f$.

Implementation. Note that our definition of a threshold signature scheme admits the trivial implementation of just using a set of k ordinary signatures. For relatively small values of n , this may very well be a perfectly adequate implementation. (Such a scheme cannot be used to implement the coin-tossing scheme, however.)

The scheme of Shoup [Sho00] is well suited to our purposes and is much more efficient than the above trivial implementation when n gets large. Each signature share is essentially the size of an RSA signature, and shares can be quite efficiently combined to obtain a completely standard RSA signature. The signature shares come with “proofs of correctness.” These correctness proofs are not much bigger than RSA signatures; however, in an efficient implementation, one would most likely omit these proofs (and their verification), and only provide them if they are explicitly requested, presumably by a party whose share combination algorithm has failed to produce a correct signature.

4.3 Threshold Coin-Tossing Scheme

In this section, we define the notion of an (n, k, t) dual-threshold coin-tossing scheme. The basic idea is that there are n parties, up to t of which may be corrupted. The parties hold *shares* of an unpredictable function F mapping the name C (which is an arbitrary bit string) of a coin to its value $F(C) \in \{0, 1\}$. The parties may generate *shares of a coin*— k coin shares are both necessary and sufficient to construct the value of the particular coin. The only requirement on k is that $t < k \leq n - t$, analogous to threshold signatures. The generation and verification of coin shares are completely non-interactive; we work in the basic system model of §2.

The Action. The dealer generates secret key shares $SK_1, \dots, SK_n$, and verification keys $VK, VK_1, \dots, VK_n$. The initial state information for party P_i consists of the secret key SK_i along with all the verification keys. The secret keys implicitly define a function F mapping names to $\{0, 1\}$.

After the dealing phase, the adversary submits *reveal requests* to the honest parties for coins of his choice. Upon such a request, party P_i outputs a *coin share* for the given coin, which it computes using SK_i .

Combining Coin Shares. The coin-tossing scheme also specifies two algorithms: a *share verification* algorithm, and a *share combining* algorithm.

- The *share verification* algorithm takes as input the name of a coin, a share on this coin from a party P_i , along with VK and VK_i , and determines if the coin share is valid.
- The *share combining* algorithm takes as input a the name C of a coin and k valid shares of C , along with (perhaps) the verification keys, and (hopefully) outputs $F(C)$.

Security Requirements. The two basic security requirements are *robustness* and *unpredictability*.

Robustness. It is computationally infeasible for an adversary to produce a name C and k valid shares of C such that the output of the share combining algorithm is not $F(C)$.

Unpredictability. An adversary's *advantage* in the following game is negligible. The adversary interacts with the honest parties as above, and at the end of this interaction, he outputs a name C that was submitted as a reveal request to fewer than $k - t$ honest parties, and a bit $b \in \{0, 1\}$. The adversary's advantage in this game is defined to be the distance from $1/2$ of the probability that $F(C) = b$. Note that if the adversary actually corrupts $f < t$ parties, the relevant threshold is still $k - t$ and not $k - f$.

Unpredictability for Sequences of Coins. The unpredictability property above implies the following more general unpredictability property that we actually need in order to analyze agreement protocols.

Consider an adversary A that interacts with the honest parties as above, but as it interacts, it makes a sequence of predictions, predicting $b_i \in \{0, 1\}$ as the value of coin C_i for $i = 1, \dots, q$ for some q . A 's predictions are interleaved with reveal requests in an arbitrary way, subject only to the restriction that at the point in time that A predicts the value of coin C_i , it has made fewer than $k - t$ reveal requests for C_i . After it predicts C_i , it may make as many reveal requests for C_i as it wishes. For $1 \leq i \leq q$, let $e_i = F(C_i) \oplus b_i$. This defines the *error vector* $(e_1, \dots, e_q)$.

The unpredictability property above implies that the error vector is *computationally indistinguishable* from a random bit-vector of length q . This means that there is no effective statistical test that distinguishes the error vector from a random vector—the important point is that we are considering statistical tests that receive *only* the test vector as input, and no additional information about A 's interaction in the above game.

A proof of this can be adapted easily from the work of Beaver and So [BS94] (omitted from this extended abstract).

Implementation. Note that an implementation of a coin-tossing scheme can be obtained from any non-interactive threshold signature scheme with the property that there is only one valid signature per message, such as the RSA-based scheme mentioned earlier [Sho00]. Then a cryptographic hash of the signature can be used as the value of the coin. It is straightforward to see that in the random oracle model, this yields a secure coin-tossing scheme. It also allows an implementation to “optimistically” skip the verification tests unless necessary.

In §6 we present also a direct implementation of a coin-tossing scheme based on the Diffie-Hellman problem.

5 Asynchronous Byzantine Agreement

5.1 Protocol ABBA

We now present our protocol ABBA, which stands for Asynchronous Binary Byzantine Agreement. As usual there are n parties $P_1, \dots, P_n$, up to t of which may be corrupted by the adversary. We denote by f the actual number of parties corrupted.

The protocol uses an $(n, n - t, t)$ threshold signature scheme $\mathcal{S}$ and an $(n, t + 1, t)$ threshold signature scheme $\mathcal{S}_0$ (see §4.2), as well as an $(n, n - t, t)$ threshold coin-tossing scheme (see §4.3). Let $F(C)$ denote the value of coin with name C .

Overview. For a given transaction identifier TID , each party P_i has an initial value $V_i \in \{0, 1\}$, and the protocol proceeds in rounds $r = 1, 2, \dots$. The first round starts with a special pre-processing step:

0. Each party sends its *initial value* to all other parties signed with an $\mathcal{S}_0$ -signature share. On receiving $2t + 1$ such votes, each party combines the signature shares of the value with the simple majority (i.e., at least $t + 1$ votes) to a threshold signature of $\mathcal{S}_0$. This value will be the value used in the first pre-vote. (This step is not necessary if the input values are accompanied by validating data.)

After that each round contains four basic steps:

1. Each party casts a *pre-vote* for a value $b \in \{0, 1\}$. These pre-votes must be *justified* by an appropriate $\mathcal{S}$ -threshold signature, and must be accompanied by a valid $\mathcal{S}$ -signature share on an appropriate message.
2. After collecting $n - t$ valid pre-votes, each party casts a *main-vote* $v \in \{0, 1, \text{abstain}\}$. As with pre-votes, these main-votes must be justified by an appropriate $\mathcal{S}$ -threshold signature, and must be accompanied by a valid $\mathcal{S}$ -signature share on an appropriate message.
3. After collecting $n - t$ valid main-votes, each party examines these votes. If all votes are for a value $b \in \{0, 1\}$, then the party *decides* b for TID , but continues to participate in the protocol for one more round. Otherwise, the party proceeds.
4. The value of coin (TID, r) is revealed, which may be used in the next round.

We now proceed with the details of the protocol given in Figure 1. We first introduce some conventions.

Recall that a message from P_i to P_j has the form $(TID, i, j, \text{payload})$, so that in specifying a message, we will only specify the *payload* if necessary; the values of TID , i , and j are implied from the context.

The pre-vote and main-vote messages have to contain a proper *justification*, which consists of threshold signatures on collected votes as follows.

Pre-Vote Justification. In round $r = 1$, party P_i 's pre-vote is the majority of the pre-processing votes from step 0. There must be at least $t + 1$ votes for the same value $b \in \{0, 1\}$ (although this b might not be unique if $n > 3t + 1$). For the justification, a party selects $t + 1$ such votes, and combines the accompanying $\mathcal{S}_0$ -signature shares to obtain an $\mathcal{S}_0$ -threshold signature on the message

$$(TID, \text{pre-process}, b).$$

In rounds $r > 1$, a pre-vote for b may be justified in two ways:

- either with an $\mathcal{S}$ -threshold signature on the message

$$(TID, \text{pre-vote}, r - 1, b);$$

we call this a *hard* pre-vote for b ;

- or with an $\mathcal{S}$ -threshold signature on the message

$$(TID, \text{main-vote}, r - 1, \text{abstain})$$

for the pre-vote $b = F(TID, r - 1)$; we call this a *soft* pre-vote for b .

Intuitively, a hard pre-vote expresses P_i 's preference for b based on evidence for preference b in round $r - 1$, whereas a soft pre-vote is just a vote for the value of the coin, based on evidence of conflicting votes in round $r - 1$. The threshold signatures are obtained from the computations in previous rounds (see below). We assume that the justification indicates whether the pre-vote is hard or soft.

Main-Vote Justification. A main-vote v in round r is one of the values $\{0, 1, \text{abstain}\}$ and, like pre-votes, accompanied by a *justification* as follows:

- If among the $n - t$ justified round- r pre-votes collected by P_i there is a pre-vote for 0 and a pre-vote for 1, then P_i 's main-vote v for round r is **abstain**. The justification for this main-vote consists of the justifications for the two conflicting pre-votes.
- Otherwise, P_i has collected $n - t$ justified pre-votes for some $b \in \{0, 1\}$ in round r , and since each of these comes with a valid $\mathcal{S}$ -signature share on the message

$$(TID, \text{pre-vote}, r, b),$$

party P_i can combine these shares to obtain a valid $\mathcal{S}$ -threshold signature on this message. Party P_i 's main-vote v in this case is b , and its justification is this threshold signature.

The protocol is shown in Figure 1.

5.2 Analysis

Theorem 1 *Assuming a secure threshold signature scheme, a secure threshold coin-tossing scheme, and a secure message authentication code, protocol ABBA solves asynchronous Byzantine agreement for $n > 3t$.*

The rest of this section outlines a proof of this theorem. We have to show *validity*, *agreement*, and *termination*.

It is straightforward to check that protocol ABBA satisfies the *validity* condition.

We prove *agreement* and *termination* assuming the adversary corrupts exactly $f = t$ parties (the modifications necessary to deal with the case $f < t$ are omitted from this extended abstract).

Fix a given TID and consider the pre-votes cast by honest parties in round $r \geq 1$. Because $n > 3t$, there will be at most one value $b \in \{0, 1\}$ that garners at least $n - 2t$ such pre-votes, and we define ρ_r to be this value (if it exists), and otherwise we say that ρ_r is undefined. We say that ρ_r is defined at the point in the game at which time sufficient pre-votes are cast.

Lemma 2 *For $r \geq 1$, the following holds (with all but negligible probability):*

- if an honest party casts or accepts a main-vote of $b \in \{0, 1\}$ in round r , then ρ_r is defined and $\rho_r = b$;
- if an honest party casts or accepts a hard pre-vote for $b \in \{0, 1\}$ in round $r + 1$, then ρ_r is defined and $\rho_r = b$;
- if an honest party casts or accepts a main-vote of **abstain** in round $r + 1$, then ρ_r is defined and $\rho_r = 1 - F(TID, r)$;

Protocol ABBA for party P_i with initial value V_i

0. PRE-PROCESSING. Generate an $\mathcal{S}_0$ -signature share on the message

$$(TID, \text{pre-process}, V_i).$$

and send a message of the form

$$(\text{pre-process}, V_i, \text{signature share})$$

to all parties.

Collect $2t + 1$ proper pre-processing messages.

Repeat the following steps 1–4 for rounds $r = 1, 2, \dots$

1. PRE-VOTE. If $r = 1$, let b be the simple majority of the received pre-processing votes.

Otherwise, if $r > 1$, select $n - t$ properly justified main-votes from round $r - 1$ and let

$$b = \begin{cases} 0 & \text{if there is a main-vote for 0,} \\ 1 & \text{if there is a main-vote for 1,} \\ F(TID, r - 1) & \text{if all main-votes are abstain.} \end{cases}$$

Produce an $\mathcal{S}$ -signature share on the message

$$(TID, \text{pre-vote}, r, b).$$

Produce the corresponding justification (see text) and send to all parties a message of the form

$$(\text{pre-vote}, r, b, \text{justification}, \text{signature share}).$$

2. MAIN-VOTE. Collect $n - t$ properly justified round- r pre-vote messages. Consider these pre-votes and let

$$v = \begin{cases} 0 & \text{if there are } n - t \text{ pre-votes for 0,} \\ 1 & \text{if there are } n - t \text{ pre-votes for 1,} \\ \text{abstain} & \text{if there are pre-votes for 0 and 1.} \end{cases}$$

Produce an $\mathcal{S}$ -signature share on the message

$$(TID, \text{main-vote}, r, v).$$

Produce the corresponding justification (see text) and send to all parties a message of the form

$$(\text{main-vote}, r, v, \text{justification}, \text{signature share}).$$

3. CHECK FOR DECISION. Collect $n - t$ properly justified main-votes of round r . If these are all main-votes for $b \in \{0, 1\}$, then *decide* the value b for TID , and continue for one more round (up to step 2). Otherwise, simply proceed.

4. COMMON COIN. Generate a *coin share* of the coin (TID, r) , and send to all parties a message of the form

$$(\text{coin}, r, \text{coin share}).$$

Collect $n - t$ shares of the coin (TID, r) , and combine these shares to get the value $F(TID, r) \in \{0, 1\}$.

Figure 1: Asynchronous Binary Byzantine Agreement

- (d) if r is the first round in which any honest party decides, then all honest parties that eventually decide, decide the same value in either round r or $r + 1$.

Proof. Omitted due to lack of space. $\square$

Agreement follows from part (d). All that remains is termination. For this, we need to show *deadlock freeness* and *fast convergence*.

Deadlock freeness is fairly straightforward. It is clear that honest parties will proceed from one round to the next, provided the adversary delivers enough messages between the honest parties. The *deadlock freeness* property follows from this observation, along with part (d) of Lemma 2, and the fact that parties who decide play along for one more round.

All that remains is *fast convergence*. Lemma 2 says that in a given round $r + 1$, for $r \geq 1$, the set of $n - t$ main-votes accepted by an honest party in step 3 contains votes for either 0 or 1, but not both. Also, such an honest party will decide in this round unless it accepts at least one main-vote of abstain. But if it does accept an abstain, then $\rho_r = 1 - F(TID, r)$. The key to showing fast termination will be to show that the value of ρ_r is determined before the coin (TID, r) is revealed.

By “ ρ_r is determined at a particular point in time,” we mean the following: There is an efficient procedure W that takes as input a transcript describing the adversary’s interaction with the system up to the given point in time, along with TID and $r \geq 1$, and outputs $w \in \{0, 1, ?\}$. Furthermore, if the output is $w \neq ?$, then if ρ_r ever becomes defined, it must be equal to w (or at least, it should be computationally infeasible for an adversary to cause this not to happen).

By “the coin (TID, r) is revealed at a particular point in time,” we mean the point in time when an honest party generates the $(n - 2t)$ -th share of the coin (TID, r) .

Lemma 3 *There is a function W that determines ρ_r , as described above, such that for all $r \geq 1$, either ρ_r is determined before coin (TID, r) is revealed, or ρ_{r+1} is determined before $(TID, r + 1)$ is revealed.*

Proof. Omitted due to lack of space. $\square$

This lemma, together with the unpredictability property of sequences of coins described in §4.3, implies that the probability that any honest party advances more than $2r + 1$ rounds is bounded by $2^{-r} + \epsilon$, where ϵ is negligible. *Fast convergence* follows immediately. Note that to make this argument rigorous, we need to be able to explicitly “predict” (as in §4.3) the desired value of the coin $(1 - \rho_r)$ that would delay termination, which is why we defined the notion of “determining” ρ_r as we did.

We remark that if the first honest parties to decide make their decision in round r , there may be others who make their decision in round $r + 1$. The “early deciders” play along for round $r + 1$, which allows the “late deciders” to decide. However, the “late deciders” do not “know” they are “late,” so they attempt to play along for round $r + 2$. What happens is that in round $r + 2$, the protocol will “fizzle out”: the “late deciders” will simply end up waiting in step 2 for $n - t$ messages that never arrive. This “fizzling out” does indeed satisfy our technical definition of termination, and is perhaps adequate for some settings; however, a more “decisive” termination can be achieved with a minor modification of the protocol (see §5.3.1).

5.3 Variations

Protocol ABBA can be modified several ways.

5.3.1 Achieving Stronger Termination

As we briefly discussed in §5.2, some parties may terminate an instance of a protocol in a rather indecisive way: although they have made a decision, they do not know that they can stop; they will wait forever. It is easy to modify protocol ABBA so that parties not only decide, but terminate in a more decisive fashion. Namely, when a party P_i decides b for TID in round r , it can combine the signature shares that it has on hand to construct an $\mathcal{S}$ -threshold signature on the message

$$(\text{main-vote}, TID, r, b).$$

It then sends this threshold signature to all parties and stops. Any other party that receives the above threshold signature, can also decide b for TID , send the this signature to all parties, and then stop.

Note that without this modification, the threshold signatures on main-votes other than abstain are actually not used by the protocol, and could be deleted.

5.3.2 Using an $(n, t + 1, t)$ Coin-Tossing Scheme

Instead of an $(n, n - t, t)$ coin-tossing scheme, one could use an $(n, t + 1, t)$ coin-tossing scheme, provided that before a party releases its share of a coin, it sends an appropriate “ready” message to all parties, and waits for $n - t$ corresponding “ready” messages from other parties. These “ready” messages do not need to be signed—the authenticity of the messages is enough. This modification increases the communication complexity of the protocol; however, an $(n, t + 1, t)$ coin can be implemented based on weaker intractability assumptions than and $(n, n - t, t)$ coin, and so the tradeoff may be worthwhile in some settings.

5.3.3 Further Optimizations

Although we have strived to make our protocol as efficient as possible, we have omitted several optimizations in order to simplify the presentation; they are described next. Some of them lead to a more flexible, “pipelined” execution of the protocol steps.

1. A party need not generate a share of the coin in round $r + 1$ if it did not accept a main-vote of abstain in round r .
2. A party need not wait for $n - t$ coin shares, unless it is going to cast a soft pre-vote, or unless it needs to later verify the justification of a soft pre-vote (it can always wait for them later if needed).
3. A party need not wait for $n - t$ pre-votes once it accepts two conflicting pre-votes, since then it is already in a position to cast a main-vote of abstain.
4. A party need not wait for $n - t$ main-votes if it has already accepted a main-vote for something other than abstain, since then it is already in a position to move to the next round; however, the decision condition should be checked before the end of the next round.

5. It is possible to collapse steps 4 and 1; however, some adjustments must be made to accommodate the threshold signature. If a party wants to make a hard pre-vote for b , he should generate signature shares on two messages that say “I pre-vote b if the coin is 0” and “I pre-vote b if the coin is 1.” If a party wants to make a soft pre-vote, he should generate signature shares on two messages that say “I pre-vote 0 if the coin is 0” and “I pre-vote 1 if the coin is 1.” This allows the parties to make soft pre-votes and reveal the coin concurrently, while also making it possible to construct the necessary main-vote justifications. This variation reduces the round and message complexity by a factor of $1/3$, at the expense of somewhat higher computational and bit complexity; it also precludes variations (1) and (2) above.

6 A Diffie-Hellman Based Threshold Coin-Tossing Scheme

In this section, we present an (n, k, t) threshold coin-tossing scheme based on the Diffie-Hellman problem. We work with a group G of large prime order q .

At a high level, our scheme works as follows. The value of a coin C is obtained by first hashing C to obtain $\tilde{g} \in G$, then raising $\tilde{g}$ to a secret exponent $x_0 \in \mathbb{Z}_q$ to obtain $\tilde{g}_0 \in G$, and finally hashing $\tilde{g}_0$ to obtain the value $F(C) \in \{0, 1\}$. The secret exponent x_0 is distributed among the parties using Shamir’s secret sharing scheme [Sha79]. Each party P_i holds a share x_i of x_0 ; its share of $F(C)$ is $\tilde{g}^{x_i}$, along with a “validity proof.” Shares of coin C can then be combined to obtain $\tilde{g}_0$ by interpolation “in the exponent.”

In more detail, we need cryptographic hash functions

$$\begin{aligned} H &: \{0, 1\}^* \rightarrow G; \\ H' &: G^6 \rightarrow \mathbb{Z}_q; \\ H'' &: G \rightarrow \{0, 1\}. \end{aligned}$$

No specific requirements are made for these hash functions, but they will be modeled as random oracles in the analysis. (H'' could actually be implemented in the standard model, e.g., by the inner product of the bit representation of the input with a random bit string, chosen once and for all by the dealer.)

In the dealing phase, the dealer selects k coefficients of a random polynomial $f(T)$ over $\mathbb{Z}_q$ of degree less than k and a random generator g of G . For $0 \leq i \leq n$, let $x_i = f(i)$ and $g_i = g^{x_i}$. Party P_i ’s secret key SK_i is x_i , and his verification key VK_i is g_i . The global verification key VK consists of a description of G (which includes q) and g .

For a general coin $C \in \{0, 1\}^*$, we let $\tilde{g} = H(C)$, and $\tilde{g}_i = \tilde{g}^{x_i}$ for $0 \leq i \leq n$. The value of the coin is $F(C) = H''(\tilde{g}_0)$.

For a given coin C , party P_i ’s share of the coin is $\tilde{g}_i$, together with a “validity proof,” i.e., a proof that $\log_{\tilde{g}} \tilde{g}_i = \log_g g_i$. This proof is the well-known interactive proof of equality of discrete logarithms (see [CP93]), collapsed into a non-interactive proof using the Fiat-Shamir heuristic [FS87]. A valid proof is a pair $(c, z) \in \mathbb{Z}_q \times \mathbb{Z}_q$, such that

$$c = H'(g, g_i, h, \tilde{g}, \tilde{g}_i, \tilde{h}), \quad (1)$$

where

$$h = g^z / g_i^c \text{ and } \tilde{h} = \tilde{g}^z / \tilde{g}_i^c.$$

Party P_i computes such a proof by choosing $s \in \mathbb{Z}_q$ at random, computing $h = g^s$, $\tilde{h} = \tilde{g}^s$, and obtaining c as in (1) and $z = s + x_i c$.

Now, for any set S of k distinct points in $\mathbb{Z}_q$, and any $\beta \in \mathbb{Z}_q$, there exist elements $\lambda_{\alpha, \beta}^S \in \mathbb{Z}_q$ for $\alpha \in S$, such that

$$\sum_{\alpha \in S} f(\alpha) \lambda_{\alpha, \beta}^S = f(\beta).$$

These λ -values are independent of $f(T)$, and can be computed from the formulas for Lagrange interpolation.

To combine a set of valid shares $\{\tilde{g}_\alpha : \alpha \in S\}$, one simply computes

$$\tilde{g}_0 = \prod_{\alpha \in S} \tilde{g}_\alpha^{\lambda_{\alpha, 0}^S}.$$

The value of the coin is then computed as $H''(\tilde{g}_0)$.

To analyze this scheme, we need to consider the following two intractability assumptions. For $g, g_0, \hat{g} \in G$, define $DH(g, g_0, \hat{g})$ to be $\hat{g}_0 = \hat{g}^{x_0}$, provided that $g_0 = g^{x_0}$. Also, define $DHP(g, g_0, \hat{g}, \hat{g}_0)$ to be 1 if $\hat{g}_0 = DH(g, g_0, \hat{g})$, and 0 otherwise.

The Computational Diffie-Hellman (CDH) assumption is the assumption that DH is hard to compute—that is, there is no efficient, probabilistic algorithm that computes DH correctly (with negligible error probability) on all inputs.

The Decisional Diffie-Hellman (DDH) assumption is the assumption that DHP is hard to compute—that is, there is no efficient, probabilistic algorithm that computes DHP correctly (with negligible error probability) on all inputs.

Theorem 4 *In the random oracle model, the above coin-tossing scheme is secure under the CDH assumption, if $k = t + 1$, and under the DDH assumption otherwise.*

Proof. Omitted due to lack of space. $\square$

References

- [BCK98] M. Bellare, R. Canetti, and H. Krawczyk, *A modular approach to the design and analysis of authentication and key exchange protocols*, Proc. 30th Annual ACM Symposium on Theory of Computing (STOC), 1998.
- [Ben83] M. Ben-Or, *Another advantage of free choice: Completely asynchronous agreement protocols*, Proc. 2nd ACM Symposium on Principles of Distributed Computing (PODC), 1983.
- [BG93] P. Berman and J. A. Garay, *Randomized distributed agreement revisited*, Proc. 23th International Symposium on Fault-Tolerant Computing (FTCS-23), 1993, pp. 412–419.
- [Boy89] C. Boyd, *Public-key cryptography and re-usable shared secrets*, Cryptography and Coding (H. Baker and F. Piper, eds.), Clarendon Press, 1989, pp. 241–246.
- [BR93] M. Bellare and P. Rogaway, *Random oracles are practical: A paradigm for designing efficient protocols*, Proc. 1st ACM Conference on Computer and Communications Security, 1993.

- [BR95] M. Bellare and P. Rogaway, *Provably secure session key distribution—the three party case*, Proc. 27th Annual ACM Symposium on Theory of Computing (STOC), 1995, pp. 57–66.
- [Bra84] G. Bracha, *An asynchronous $[(n - 1)/3]$ -resilient consensus protocol*, Proc. 3rd ACM Symposium on Principles of Distributed Computing (PODC), 1984, pp. 154–162.
- [BS94] D. Beaver and N. So, *Global, unpredictable bit generation without broadcast*, Advances in Cryptology: EUROCRYPT '93 (T. Helleseeth, ed.), Lecture Notes in Computer Science, vol. 765, Springer, 1994.
- [Can96] R. Canetti, *Studies in secure multiparty computation and applications*, Ph.D. thesis, Weizmann Institute of Science, 1996.
- [CD89] B. Chor and C. Dwork, *Randomization in Byzantine agreement*, Randomness and Computation (S. Micali, ed.), Advances in Computing Research, vol. 5, JAI Press, 1989, pp. 443–497.
- [CG99] R. Canetti and S. Goldwasser, *An efficient threshold public-key cryptosystem secure against adaptive chosen-ciphertext attack*, Advances in Cryptology: EUROCRYPT '99 (J. Stern, ed.), Lecture Notes in Computer Science, vol. 1592, Springer, 1999, pp. 90–106.
- [CH89] R. A. Croft and S. P. Harris, *Public-key cryptography and re-usable shared secrets*, Cryptography and Coding (H. Baker and F. Piper, eds.), Clarendon Press, 1989, pp. 189–201.
- [CKS00] C. Cachin, K. Kursawe, and V. Shoup, *Random oracles in Constantinople: Practical asynchronous Byzantine agreement using cryptography*, To appear in Cryptology ePrint Archive, <http://eprint.iacr.org>, 2000.
- [CP93] D. Chaum and T. P. Pedersen, *Wallet databases with observers*, Advances in Cryptology: CRYPTO '92 (E. F. Brickell, ed.), Lecture Notes in Computer Science, vol. 740, Springer-Verlag, 1993, pp. 89–105.
- [CR93] R. Canetti and T. Rabin, *Fast asynchronous Byzantine agreement with optimal resilience*, Proc. 25th Annual ACM Symposium on Theory of Computing (STOC), 1993, pp. 42–51.
- [CT96] T. D. Chandra and S. Toueg, *Unreliable failure detectors for reliable distributed systems*, Journal of the ACM **43** (1996), no. 2, 225–267.
- [Des88] Y. Desmedt, *Society and group oriented cryptography: A new concept*, Advances in Cryptology: CRYPTO '87 (C. Pomerance, ed.), Lecture Notes in Computer Science, vol. 293, Springer, 1988, pp. 120–127.
- [DF90] Y. Desmedt and Y. Frankel, *Threshold cryptosystems*, Advances in Cryptology: CRYPTO '89 (G. Brassard, ed.), Lecture Notes in Computer Science, vol. 435, Springer, 1990, pp. 307–315.
- [DS83] D. Dolev and H. R. Strong, *Authenticated algorithms for Byzantine agreement*, SIAM Journal on Computing **12** (1983), no. 4, 656–666.
- [FLP85] M. J. Fischer, N. A. Lynch, and M. S. Paterson, *Impossibility of distributed consensus with one faulty process*, Journal of the ACM **32** (1985), no. 2, 374–382.
- [FM97] P. Feldman and S. Micali, *An optimal probabilistic protocol for synchronous Byzantine agreement*, SIAM Journal on Computing **26** (1997), no. 4, 873–933.
- [FS87] A. Fiat and A. Shamir, *How to prove yourself: Practical solutions to identification and signature problems*, Advances in Cryptology: CRYPTO '86 (A. M. Odlyzko, ed.), Lecture Notes in Computer Science, vol. 263, Springer, 1987, pp. 186–194.
- [GMR88] S. Goldwasser, S. Micali, and R. L. Rivest, *A digital signature scheme secure against adaptive chosen-message attacks*, SIAM Journal on Computing **17** (1988), no. 2, 281–308.
- [Lyn96] N. A. Lynch, *Distributed algorithms*, Morgan Kaufmann, San Francisco, 1996.
- [MS95] S. Micali and R. Sidney, *A simple method for generating and sharing pseudo-random functions, with applications to clipper-like key escrow systems*, Advances in Cryptology: CRYPTO '95 (D. Coppersmith, ed.), Lecture Notes in Computer Science, vol. 963, Springer, 1995, pp. 185–196.
- [NPR99] M. Naor, B. Pinkas, and O. Reingold, *Distributed pseudo-random functions and KDCs*, Advances in Cryptology: EUROCRYPT '99 (J. Stern, ed.), Lecture Notes in Computer Science, vol. 1592, Springer, 1999.
- [Rab83] M. O. Rabin, *Randomized Byzantine generals*, Proc. 24th IEEE Symposium on Foundations of Computer Science (FOCS), 1983, pp. 403–409.
- [Rab98] T. Rabin, *A simplified approach to threshold and proactive RSA*, Advances in Cryptology: CRYPTO '98 (H. Krawczyk, ed.), Lecture Notes in Computer Science, vol. 1462, Springer, 1998.
- [Rei96] M. K. Reiter, *A secure group membership protocol*, IEEE Transactions on Software Engineering **22** (1996), no. 1, 31–42.
- [Sha79] A. Shamir, *How to share a secret*, Communications of the ACM **22** (1979), 612–613.
- [Sho99] V. Shoup, *On formal models for secure key exchange*, Research Report RZ 3120, IBM Research, 1999, revision 4, available from <http://www.shoup.net>.
- [Sho00] V. Shoup, *Practical threshold signatures*, Advances in Cryptology: EUROCRYPT 2000 (B. Preneel, ed.), Lecture Notes in Computer Science, Springer, 2000, (to appear).
- [Tou84] S. Toueg, *Randomized Byzantine agreements*, Proc. 3rd ACM Symposium on Principles of Distributed Computing (PODC), 1984, pp. 163–178.